

# CS & IT ENGINEERING



## Operating System

Process Synchronization

Lecture - 04

By- Vishvadeep Gothi sir



# Recap of Previous Lecture



**Topic**

**Two-Process Solution for Critical Section**

**Topic**

**Synchronization Hardware**

**Topic**

**Test-And-Set(), Swap()**



# Topics to be Covered



Topic

Semaphore

Topic

Wait() & Signal()

Topic

Counting Semaphore

starvation :-

indefinite wait to execute critical section

or

to execute further

## Solution 1

✓ starvation

Boolean lock=false;

lock = True

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

## Solution 2

✓ starvation

```
int turn=0;
```

```
while(true)
{
    while(turn!=0);
    CS
    turn=1;
    RS;
}
```

```
while(true)
{
    while(turn!=1);
    CS
    turn=0;
    RS;
}
```

## Peterson's Solution

```
Boolean Flag[2];
int turn;
```

```
while(true) {
    Flag[0]=true;
    turn=1;
    while(Flag[1] && turn==1);
        CS
    Flag[0]=False;
    RS;
}
```

✗ starvation

```
while(true){
    Flag[1]=true;
    turn=0;
    while(Flag[0] && turn==0);
        CS
    Flag[1]=False;
    RS;
}
```





## Topic : Synchronization Hardware



1. TestAndSet()
2. Swap()





## Topic : TestAndSet()

Returns the current value flag and sets it to true.



## Topic : TestAndSet()



✓ starvation

```
Boolean Lock=False;
```

```
boolean TestAndSet(Boolean *trg){
```

```
boolean rv = *trg;
```

```
*trg = True;
```

```
Return rv;
```

```
}
```

```
while(true)
```

```
{
```

```
while(TestAndSet(&Lock));
```

```
CS
```

```
Lock=False;
```

```
}
```



## Topic : Swap()



✓ starvation

```
Boolean Key;           //Local
Boolean Lock=False;    //Shared

void Swap(Boolean *a, Boolean *b)
{
    boolean temp = *a;
    *a=*b;
    *b=temp;
}
```

```
while(true){
    Key = True;
    while (key==True)
        Swap(&Lock, &Key);

    CS
    Lock=False;
    RS
}
```





## Topic : Synchronization Tool

1. Semaphore
- ✗ 2. Monitor

→ not dependent on platform  
or  
hardware



## Topic : Synchronization Tool

- <sup>non-negative</sup> Integer value which can be accessed using following functions only
  1. wait() / P() / Degraded()
  2. signal() / V() / Upgrade() } atomic functions



## Topic : wait() & signal()

```
wait(S)
```

```
{
```

```
    while(S<=0);
```

```
    S--;
```

```
}
```

```
signal(S)
```

```
{
```

```
    S++;
```

```
}
```

*wait(S) can not be successful if  $S=0$*



## Topic : Types of Semaphore

| Binary Semaphore                                 | Counting Semaphore                                             |
|--------------------------------------------------|----------------------------------------------------------------|
| <p>↓</p> <p>Can have value<br/>either 0 or 1</p> | <p>Can have value</p> <p>↓</p> <p>0, 1, 2, 3, 4, 5, 6, ...</p> |





## Topic : Types of Semaphore

| Binary Semaphore                                                                          | Counting Semaphore                                                     |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| It is used to implement the solution of critical section problems with multiple processes | It is used to control access to a resource that has multiple instances |

↓  
mutual exclusion



## Topic : Critical Section Solution

binary semaphore  
 $S = 1$

$P_0, P_1$

while(True)

```
{  
    wait(S) → entry section  
    C.S.  
    signal(s) → Exit section  
}
```

$S = 1 \ 0 \ 1 \ 0 \ 1$

if  
counting semaphore  $S = 2$

$P_0 \ P_1 \ P_2 \ P_3$

```
while (True)  
{  
    wait(S)  
    C.S.  
    signal(S)  
}
```

race condition possible

no mutual exclusion

as 2 processes can enter into CS together.

Binary semaphore  $s = 1 \Rightarrow \text{signal}(s) \Rightarrow$  successfully completed  
and  $s$  remains 1

Binary semaphore  $s = 0 \Rightarrow \text{wait}(s) \Rightarrow$  not-successfully completed,  
stuck into infinite loop



#Q. If a counting semaphore S is initialized with value 2. There are 4 processes P1, P2, P3 and P4. Maximum number of processes which can be in critical section together if all process running the following code?

P1 P2 P3 P4  
while(True)  
{  
  wait(S)  
  C.S.  
  signal(s)  
}

$S = 2 \neq 0$

Ans = 2



#Q. If a counting semaphore S is initialized with value 2. There are 4 processes P1, P2, P3 and P4. Maximum number of processes which can run critical section if all process running the following code?

```
while(True)
{
    wait(S)
    C.S.
    signal(s)
}
```

*all processes*

- a) 1
- b) 2
- c) 3
- ☒ d) 4

[NAT]



#Q. Consider a counting semaphore S, initialized with value 21. What should be the value of S after executing 14 times P() and 7 time V() function on S?

$$\begin{array}{r} 21 \\ - 14 \\ + 7 \\ \hline 14 \end{array}$$

Ans  $\Rightarrow 14$

#Q. Consider a semaphore S, initialized with value 37. Which of the following options gives the final value of S=12?

**A** Execution of 22 P() and 15 V()  $37 - 22 + 15 = 30$

**B** ✓ Execution of 25 P()  $37 - 25 = 12$

**C** ✓ Execution of 33 P() and 8 V()  $37 - 33 + 8 = 12$

**D** ✓ Execution of 31 P() and 6 V()  $37 - 31 + 6 = 12$



#Q. Consider a binary semaphore S, initialized with value 1. Consider 10 processes P1, P2 .... P10. All processes have same code as given below but, one process P10 has signal(S) in place of wait(S). If all processes to be executed only once, then maximum number of processes which can be in critical section together ?

```
process
{
    wait(S)
    C.S.
    signal(s)
}
```

Ans = 3





## Topic : Solution



P1, P2, ....., P9

```
process
{
    wait(S)
    C.S.
    signal(s)
}
```

P10

```
process
{
    signal(S)
    C.S.
    signal(s)
}
```

$$S = 1 \neq 0$$

$P_1 \Rightarrow \text{wait}(s) \Rightarrow \text{into CS}$   
 $P_{10} \Rightarrow \text{signal}(s) \Rightarrow \text{into CS}$   
 $P_2 \Rightarrow \text{wait}(s) \Rightarrow \text{into CS}$

## [MCQ]



#Q. Consider a binary semaphore S, initialized with value 1. Consider 10 processes P1, P2 .... P10. All processes have same code as given below but, one process P10 has signal(S) in place of wait(S). If all processes to be executed only once, then maximum number of processes which can be in critical section together ?

```
while(True)
{
    wait(S)
    C.S.
    signal(s)
}
```

Ans = 10



## Topic : Solution



**P1, P2, ....., P9**

```
while(True)
{
    wait(S)
    C.S.
    signal(s)
}
```

**P10**

```
while(True)
{
    signal(S)
    C.S.
    signal(s)
}
```

$S = 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1$

$P1 \Rightarrow CS$

$P10 \Rightarrow$

$P2 \Rightarrow CS$

$P10 \Rightarrow$

$P3 \Rightarrow CS$

$P10 \Rightarrow$

all processes in CS





## 2 mins Summary

**Topic**

**Semaphore**

**Topic**

**Wait() & Signal()**

**Topic**

**Counting Semaphore**



**Happy Learning**

**THANK - YOU**